

CS 550 Operating Systems, Spring 2026
Homework Assignment 3 (HW3)

Out: 3/1/2026 Sun.

Due: 3/8/2026 Sun. 23:59:59

Full points: 100

Note:

- There are 4 questions, each on a separate page.
- Read the submission instructions on the last page carefully and prepare the submission accordingly.

1. (36 points) Suppose the calls to `pthread_create()` and `pthread_join()` in the following code always succeed:

```
01  int sum = 0;
02
03  void *A_work(void *arg) {
04      sum += 1;
05      sum += 2;
06      return NULL;
07  }
08
09  void *B_work(void *arg){
10      sum += 3;
11      sum += 4;
12      return NULL
13  }
14
15  int main() {
16      pthread_t tA, tB;
17
18      pthread_create(&tA, NULL, A_work, NULL);
19      pthread_create(&tB, NULL, B_work, NULL);
20
21      pthread_join(tA, NULL);
22      pthread_join(tB, NULL);
23
24      printf("Total Sum: %d\n", sum);
25      return 0;
30  }
```

As we know, each of the variable increment statements in lines 4, 5, 10, and 11 is compiled to three instructions:

```
load (the variable value from memory to a register)
add  (the desired value to the register)
store (the register value back to memory)
```

One execution sequence that makes the desired value for sum (i.e., 10) can be:

```
L4.load → L4.add → L4.store → L5.load → L5.add → L5.store → L10.load →
L10.add → L10.store → L11.load → L11.add → L11.store
```

where, for example, “L4.load” means the `load` instruction of line 4 statement and “L10.add” represents the `add` instruction of line 10 statement.

- (1) (6 points) Fill out the blanks in the following execution sequence such that it also makes “sum” 10.

```
(blank-1) → (blank-2) → (blank-3) → L4.load → L4.add → L4.store → (blank-4)
→ (blank-5) → (blank-6) → L5.load → L5.add → L5.store
```

- (2) (9 points) Fill out the blanks in the following execution sequence such that it makes “sum” 9.

L10.load → L10.add → (blank-1) → (blank-2) → L11.add → L4.load → (blank-3)
→ (blank-4) → (blank-5) → (blank-6) → L5.add → L5.store

(3) (9 points) Fill out the blanks in the following execution sequence such that it makes “sum” 8.

L4.load → L4.add → (blank-1) → (blank-2) → L10.add → (blank-3) → (blank-4)
→ L11.add → L5.load → L5.add → (blank-5) → (blank-6)

(4) (12 points) Fill out the blanks in the following execution sequence such that it makes “sum” 5.

L4.load → L4.add → L10.load → L10.add → (blank-1) → (blank-2) → (blank-3)
→ (blank-4) → L5.load → (blank-5) → (blank-6) → L11.store

2. (21 points) For the following code, assume `pthread_create()` and `pthread_join()` always succeed. Also assume `printf()` prints each line atomically (a full line appears without interleaving characters).

```
void* worker(void* arg) {
    int *y = (int*)arg;
    *y += 10;
    printf("W:%d\n", *y);
    return (void *) (*y);
}

pthread_t start_one() {
    pthread_t t;
    int x = 7;
    pthread_create(&t, NULL, worker, &x);
    printf("S:%d\n", x);
    return t;
}

int main() {
    pthread_t t = start_one();
    void *ret;
    pthread_join(t, &ret);
    printf("M:%d\n", (int)ret);
    return 0;
}
```

Question: Determine each statement below as Possible or Impossible for an execution of this program, and justify your answer.

- (1) The output contains a line “W:17”.
- (2) The output contains a line “W:7”.
- (3) The output contains a line “S:17”.
- (4) The output contains a line “S:7”.
- (5) The output contains a line “M:17”.
- (6) The output contains a line “M:7”.

3. (20 points) Assume three threads $T1$, $T2$ and $T3$ use three mutexes A , B , and C in the following way:

```
// Thread T1:
lock(A);
lock(B);
unlock(B);
unlock(A);

// Thread T2:
lock(B);
if (cond) {
    lock(C);
    unlock(C);
}
unlock(B);

// Thread T3:
lock(C);
lock(A);
unlock(A);
unlock(C);
```

Assume `cond` is either always true or always false (but you don't know which), and threads may be preempted at any point.

Question: For each case, determine whether deadlock is possible:

- `cond = false`
- `cond = true`

If deadlock is possible, give one deadlocking “holds / waits-for” scenario.

4. (23 points) Section 28.11 in textbook chapter 28 (<https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf>) introduces an hardware instruction `FetchAndAdd` and uses it to implement a lock that has built-in fairness (i.e., ticket lock, section 28.12).

Read chapter 28.11 carefully and use the `FetchAndAdd` instruction to implement a simple spin lock (like the ones implemented using `TestAndSet` and `CompareAndSwap` instructions). Your answer should show the code for the lock structure definition and the functions for initialization, lock, and unlock.

Submission instructions

1. Type your answers using the text editor you like. Remember to include the index number of each question.
2. Export the file to PDF format.
3. Name the PDF file based on your BU email ID. For example, if your BU email is “abc@binghamton.edu”, then the PDF file should be named as “hw3_abc.pdf”.
4. Failure to follow the above instructions will lead to 5 points off.
5. Submit the PDF file to Brightspace before the deadline.